

SOC IP Governance Automation

Technical Report: Architecture, Features, Operations, Efficiency, Pros & Cons

Date: 21 February 2026
Classification: Confidential

Prepared for operational handover and governance review

1. Executive Summary

The SOC IP Governance Automation solution is a Python + Streamlit platform designed to standardize passive IP governance operations. It transforms raw multiline IP inputs into validated, enriched, and approval-ready records while preserving auditability through SQLite persistence and controlled updates to the master blocking sheet.

The platform minimizes manual analyst effort by automating repetitive checks such as IPv4/IPv6 validation, deduplication, whitelist filtering, master-sheet comparison, and AbuseIPDB enrichment. It provides a guided approval workflow and optional notification integration, enabling faster decision cycles and improved consistency in SOC operations.

2. Scope and Operating Mode

This implementation is intentionally passive. It does not perform automated firewall pushes or Cloudflare API blocking actions. Instead, it focuses on controlled governance, analyst review, and structured documentation updates.

3. System Architecture

Core architecture is modular. The Streamlit UI orchestrates a processing pipeline and delegates specialized tasks to domain modules. Configuration is centralized and supports environment-variable overrides for operational flexibility.

Module	Primary Responsibility
app.py	UI, workflow orchestration, session state, approvals actions
ip_validator.py	IPv4/IPv6 validation, extraction, deduplication, summary generation
whitelist.py	Whitelist load + exact/CIDR filtering to prevent trusted-IP blocking
master_sheet.py	Load authoritative blocked IP set, append/update/delete master CSV rows
abuseipdb.py	AbuseIPDB check API integration with retries and rate-limit handling
database.py	SQLite initialization, scan-result and threat upserts, query/update operations
email_notifier.py	Approval email subject/body generation, SMTP/Gmail API send pipeline
gmail_auth.py	Gmail OAuth session/authentication helper functions
config.py	Runtime config resolution from env/config.ini with safe defaults
run_dashboard.sh + ngrok_tunnel.py	Single-command launch + static ngrok endpoint exposure

4. End-to-End Processing Workflow

- 1 Analyst pastes multiline IP input into Streamlit tab.
- 2 System extracts valid IPv4/IPv6 values and removes malformed entries.
- 3 Duplicates are removed while preserving input order.
- 4 Whitelist filtering removes exact-match and CIDR-covered trusted IPs.
- 5 Authoritative master blocking sheet is loaded; already-blocked IPs are excluded.
- 6 Remaining IPs are enriched via AbuseIPDB /check API.
- 7 Each scan result is upserted into SQLite for audit and later review.
- 8 Threat threshold logic identifies malicious candidates for approval workflow.
- 9 Detected threats are persisted with default status Pending.
- 10 Approved entries are appended to master sheet in required strict row format.

5. Feature Set (Operational)

- **Input hygiene:** strict regex-based IPv4/IPv6 checks and invalid-line isolation.
- **Data quality:** duplicate removal + summary metrics per run.
- **Safety control:** whitelist support (individual IP + CIDR blocks).
- **False-positive reduction:** master-sheet filtering avoids duplicate block requests.
- **Threat enrichment:** AbuseIPDB score, country, ISP metadata capture.
- **Persistence:** scan results and threat records stored in SQLite with upserts.
- **Approval workflow:** tracked status + editable PATH values + send test/approval email.
- **Master governance:** append/update/delete row operations for controlled lifecycle.
- **Launch automation:** single-command startup for Streamlit + ngrok exposure.
- **Static public access:** fixed ngrok domain configured for consistent sharing URL.

6. Reliability and Error Handling

The implementation includes targeted resilience controls: (a) request retry/backoff for API failures and HTTP 429 rate-limits, (b) fail-safe handling for malformed API payloads, (c) database upsert semantics to avoid duplicate key failures, and (d) controlled ngrok startup diagnostics for authentication and endpoint-conflict scenarios.

7. Pros and Cons

7.1 Advantages

- Significantly reduces repetitive analyst effort (validation, filtering, enrichment).
- Enforces consistent decision data before approvals (score + geo + ISP + path).
- Improves traceability through SQLite records and standardized CSV output format.
- Lower operational risk via whitelist and already-blocked filtering controls.
- Minimal infrastructure footprint; runs on standard Python + Streamlit stack.
- Easy onboarding with single-launch script and documented runbook.

7.2 Limitations / Trade-offs

- Passive model only: no direct automated firewall/Cloudflare API enforcement.
- Quality of detection depends on external AbuseIPDB API availability and accuracy.
- CSV-based master sheet can be fragile under concurrent manual edits.
- Static ngrok endpoint can conflict if same domain/session is active elsewhere.
- Email workflow relies on external provider auth state (SMTP or Gmail OAuth).
- No built-in role-based access controls inside the Streamlit interface.

8. Efficiency and Time-Saving Impact

Compared with manual SOC handling, the platform compresses multiple discrete tasks into one guided workflow. Typical manual activities (validation, duplicate checks, whitelist comparison, spreadsheet lookup, API lookup, and report formatting) are converted into deterministic pipeline stages with immediate summary metrics.

Process Step	Manual Effort (typical)	Automated Behavior	Efficiency Impact
Input validation	Line-by-line analyst review	Regex parsing + invalid extraction	High reduction
Duplicate detection	Manual visual scan	Order-preserving dedupe	High reduction
Whitelist check	Cross-reference static lists	Exact + CIDR filtering	High reduction
Threat enrichment	Manual portal checks	Automated API calls + persistence	Very high reduction
Approval packaging	Manual collation and email drafting	Template-based workflow	Medium-high reduction
Master update	Spreadsheet manual entries	Structured append/update/delete	Medium reduction

9. Ease of Adoption and Operational Usability

The system is lightweight to adopt because it uses familiar SOC artifacts (CSV sheets, straightforward thresholding, and analyst approvals) while still adding modern controls. The one-click launch script further lowers onboarding friction for shift teams and support personnel.

10. Technical Deep-Dive Highlights

10.1 Validation Engine

Regex patterns are strict for IPv4 octet range and practical for compressed/full IPv6 forms. Input lines are sanitized and length-trimmed to avoid pathological entries.

10.2 Abuse Intelligence Integration

API calls include timeout configuration, bounded retries, and progressive backoff. HTTP 429 is explicitly handled to reduce failed checks under rate pressure.

10.3 Data Persistence Model

SQLite uses primary-key upserts for both `scan_results` and `detected_threats`, ensuring idempotent updates and consistent record refresh on repeated scans.

10.4 Governance Record Management

Master CSV append enforces strict row schema including date/time, reason, and shift-tagged notes for predictable downstream audit interpretation.

10.5 Email and Approval Control

HTML email generation standardizes communication payload and includes operational columns (score, country, ISP, path, reason, approver, shift).

10.6 Config Strategy

Environment variables override `config.ini` values, enabling safe deployment-level customization without code changes.

10.7 Launch Automation and Exposure

`run_dashboard.sh` verifies runtime prerequisites, starts Streamlit in managed mode, injects ngrok token from secure local source, and invokes static-domain tunneling logic.

11. Risk Considerations and Mitigations

- **Third-party dependency risk:** External API/Tunnel outages can impact enrichment/public access. **Mitigation:** local dashboard remains available; explicit failure messages guide remediation.
- **Credential management risk:** Mismanaged tokens/keys can block operation. **Mitigation:** local token file + `.gitignore` + configuration separation.
- **Data integrity risk:** CSV manual edits can introduce format inconsistencies. **Mitigation:** programmatic append/update functions and strict parser logic.
- **Endpoint conflict risk:** static ngrok domain may already be active elsewhere. **Mitigation:** explicit conflict detection and dashboard recovery path.

12. Conclusion

SOC IP Governance Automation provides a practical balance between operational speed and governance control. It meaningfully reduces manual overhead, improves consistency and traceability, and creates a structured path from raw IOC intake to approval-ready action. With minor governance discipline around ngrok endpoint ownership and credentials, the solution is efficient, scalable for day-to-day SOC operations, and easy for shift teams to adopt.